

Operator Review & Access Control

Two changes to the Admin Command Center — why, what to build, and the endpoints behind them

This covers two changes to how the Admin Command Center works, both already built and tested on the backend. It explains why each one matters, exactly what needs to change on the dashboard, and the full request/response shape of every endpoint involved — everything needed to wire it up without guessing.

Why this update

1. AI should propose alerts, not decide to send them.

Until now, the moment a sensor reading crossed into “high” risk, the backend immediately sent a real SMS + voice alert to every subscriber in that region — no person ever saw it coming. A single noisy reading (a sensor glitch, a momentary spike, a splashed sensor during testing) could fire a real alert with no way to stop it. The fix: a sensor crossing into “high” now creates a **pending alert** and texts every admin with a phone on file. An operator can approve, edit the wording and send, or reject it with a reason — but if nobody responds within 15 minutes, it sends on its own anyway. This is deliberately fail-open: an occasional false alarm is a better outcome than silently missing a real flood because no one was watching the dashboard.

2. Only an admin should be able to create an admin account.

The old signup form was open to anyone who found the API — protected only by a shared code that, once known, let anyone register as an admin. That makes an admin console effectively public. Now, only the very first account on a fresh deployment can be self-registered (and that path locks permanently the moment one admin exists); every admin after that is created by an existing admin from inside the dashboard. There is no public signup form anymore.

What to build on the frontend

Six items below. The first three are new; the rest are fixes to what's already there, found while testing the current dashboard end-to-end.

1

Pending Alerts panel

Why: This is the actual safety mechanism — without a visible queue, "an operator can review it" is not a real workflow, just a theoretical one.

What to build:

- A new tab (alongside Priority Queue / Incident Map / Assistance / Response Center) showing every alert awaiting review: region, the rainfall/river reading and risk score that triggered it, how long until it auto-sends, and three actions — **Approve**, **Edit & Send**, **Reject** (reject requires picking a reason: false positive / sensor fault / already resolved / duplicate / other, plus optional notes).
- A persistent badge (top bar or sidebar) showing the pending count — this can't be something an operator only notices by clicking into the tab.
- A read-only history view of past pending alerts (approved/rejected/auto-sent), showing who acted and when — useful for spotting "this sensor throws a lot of false positives" patterns over time.

Endpoints used: GET /api/admin/alerts/pending, GET /api/admin/alerts/pending/history, POST .../approve, POST .../edit-and-send, POST .../reject, GET /api/admin/dashboard/stats (pending_alerts count)

2

Sensor / Device view

Why: The dashboard currently only shows the 10 static demo cities in regions.json. Operators reviewing a pending alert need to see the actual live sensor behind it — is this device normally reliable, when did it last report, is it still online — to judge a real flood from a sensor glitch.

What to build:

- A panel listing every registered device: location, online/offline status, and its most recent reading (rainfall, river level, computed risk) with a timestamp.
- Worth linking directly from a pending alert's row ("view this device's recent readings") rather than only as a separate page.

Endpoints used: GET /api/admin/devices

3

Manage Admins panel (replaces Sign Up)

Why: Public self-service signup is being removed (see "Why" above) — this is its replacement.

What to build:

- Remove the Sign Up tab/form from the admin access screen entirely. Only Login remains as a public entry point.
- Add a "Manage Admins" panel (Settings is a natural home) for a logged-in admin to see every admin account and add a new one — name, email, password, and phone number (phone is what lets that admin receive the pending-alert SMS nudge, so surface it as more than an afterthought field).
- The new admin is *not* logged in automatically — they log in themselves with the password they were given directly (Slack, in person, etc.). Handing the inviter a working session for someone else's account would be its own small security smell, so the create-admin response deliberately returns no token.

Endpoints used: GET /api/admin/admins, POST /api/admin/admins

4

Fix: no logout exists anywhere

Why: Grepping the whole frontend turned up zero references to logout — the session token is written to localStorage on login and never cleared. Anyone with access to that browser stays authenticated indefinitely; the backend's logout endpoint has never been called by anything.

What to build:

- Add a visible Sign Out control (top bar / sidebar) that calls the logout endpoint and clears the stored token, returning to the login screen.

Endpoints used: POST /api/admin/logout (already exists, already correct — just never wired up)

5

Fix: Response Center says nothing was sent, when it was

Why: The “Prepare Community Warning” confirmation reads: “Connect the notification endpoint when the backend is ready.” That's stale — the button already calls a fully working endpoint that sends real SMS/voice. Telling the operator nothing happened when something did risks a double-send.

What to build:

- Update the confirmation copy to reflect what actually happened — e.g. show the real sms_status/voice_status the endpoint already returns (sent / simulated / no_recipients / failed) instead of the placeholder line.

Endpoints used: POST /api/admin/incidents/{id}/response (already exists — this is copy-only, no new endpoint)

6

Fix or remove: the language selector in Response Center does nothing

Why: Selecting Swahili/Somali/Arabic only flips text alignment to RTL in the preview — the typed message is never translated, and the language isn't sent to the backend at all. It looks functional but isn't, and the list is missing French and Amharic, which the rest of the project already supports.

What to build:

- Either wire it to something real, or remove it until it is — a control that visibly does nothing is worse for trust during a demo than not having it.

Endpoints used: No backend endpoint currently accepts a language override on a response send — flag if this is wanted and it can be added.

Backend API reference

Base URL (local dev): `http://localhost:8000`. Every endpoint below requires `Authorization: Bearer <token>` (from `/api/admin/login`) unless marked public. All bodies are JSON.

Admin accounts

POST `/api/admin/admins`

AUTH REQUIRED (Bearer token)

Creates another admin account. Only callable by an already-authenticated admin — this is the invite flow that replaces public signup.

REQUEST BODY

```
{
  "name": "Habiba",
  "email": "habiba@afrishield-command.com",
  "password": "supersecret1",
  "phone_number": "+2348012345678"
}
```

RESPONSE

```
{
  "id": "2b2f0b5f...",
  "name": "Habiba",
  "email": "habiba@afrishield-command.com",
  "phone_number": "+2348012345678",
  "created_at": "2026-09-14T08:10:32Z"
}
```

- 201 Created. No token returned — the new admin logs in themselves.
- password must be ≥ 8 chars (422 otherwise).
- 409 if the email is already registered.
- phone_number is optional but needed for pending-alert SMS notifications.

GET `/api/admin/admins`

AUTH REQUIRED (Bearer token)

Lists every admin account — for the Manage Admins panel.

RESPONSE

```
[
  {
    "id": "91756080...", "name": "Matthias",
    "email": "matthias@aviet.aero",
    "phone_number": null,
    "created_at": "2026-09-14T07:39:05Z"
  }
]
```

POST**/api/admin/signup****PUBLIC — no auth required**

Bootstrap only: creates the very first admin account on a fresh deployment. No longer used by the dashboard once at least one admin exists — kept only for a brand-new install.

- 403 the instant any admin account already exists — this is permanent, not a one-time unlock.
- Requires a shared `signup_code` (`ADMIN_SIGNUP_CODE` in `backend/.env`).

Pending alerts (operator review)

GET**/api/admin/alerts/pending****AUTH REQUIRED (Bearer token)**

Alerts currently awaiting operator review, oldest first.

RESPONSE

```
[
  {
    "id": "7590fe0d...", "location_name": "Lagos, Nigeria",
    "risk_level": "high", "risk_score": 0.96,
    "rainfall_mm_24h": 95.0, "river_level_m": 3.9,
    "created_at": "2026-09-14T08:10:46Z",
    "auto_send_at": "2026-09-14T08:25:46Z",
    "status": "pending", "reviewed_by": null,
    "reviewed_at": null, "review_notes": null,
    "reject_reason": null, "alert_log_id": null
  }
]
```

- `auto_send_at` is when this fires on its own if nobody acts — 15 minutes after `created_at`.
- Show a countdown against `auto_send_at` so the operator sees the clock is real.

GET**/api/admin/alerts/pending/history****AUTH REQUIRED (Bearer token)**

Every pending alert ever created, newest first, regardless of outcome — the audit trail.

RESPONSE

Same shape as above, but `status` is one of `pending` / `approved` / `rejected` / `auto_sent`, and `reviewed_by` / `reject_reason` are filled in once resolved.

POST /api/admin/alerts/{id}/approve

AUTH REQUIRED (Bearer token)

Sends the alert right now with its original auto-generated message, instead of waiting out the timer.

REQUEST BODY

```
{
  "notes": "Confirmed with a local contact."
}
```

RESPONSE

The updated pending-alert record (status becomes "approved", alert_log_id filled in).

- notes is optional.
- 409 if this alert is no longer pending (already acted on, or already auto-sent).

POST /api/admin/alerts/{id}/edit-and-send

AUTH REQUIRED (Bearer token)

Same as approve, but with the operator's own wording instead of the auto-generated message.

REQUEST BODY

```
{
  "message": "URGENT (verified by field team): move to higher ground immediately.",
  "notes": "Rewrote after a call with the community leader."
}
```

RESPONSE

Same shape as approve's response.

POST /api/admin/alerts/{id}/reject

AUTH REQUIRED (Bearer token)

Dismisses the pending alert — it never sends.

REQUEST BODY

```
{
  "reason": "sensor_fault",
  "notes": "Device reads high after being splashed during testing."
}
```

RESPONSE

The updated pending-alert record (status becomes "rejected").

- reason must be one of: false_positive, sensor_fault, already_resolved, duplicate, other (422 otherwise).

Devices

GET**/api/admin/devices****AUTH REQUIRED (Bearer token)**

Every registered sensor with its most recent reading and an online/offline status.

RESPONSE

```
[
  {
    "device_id": "esp32-demo-01",
    "location_name": "Lagos, Nigeria",
    "latitude": 6.5244, "longitude": 3.3792,
    "status": "online",
    "last_reading": {
      "rainfall_mm_24h": 95.0, "river_level_m": 3.9,
      "risk_level": "high", "risk_score": 0.96,
      "received_at": "2026-09-14T08:10:46Z"
    }
  }
]
```

- status is "never_reported" if the device has no logged reading yet, "offline" if its last reading is older than 10 minutes, otherwise "online".
- last_reading is null when status is "never_reported".

Dashboard stats (changed)

GET**/api/admin/dashboard/stats****AUTH REQUIRED (Bearer token)**

Unchanged endpoint, one new field.

RESPONSE

```
{
  "total_reports": 12, "critical_or_high_priority": 3,
  "assistance_needed": 2, "resolved": 5,
  "pending_alerts": 1,
  "by_status": { "new": 4, "assigned": 3, ... }
}
```

- pending_alerts is new — drive the badge in feature #1 from this.

Known limitations (stated honestly)

The 15-minute auto-send timer lives inside the running backend process. If the server restarts while an alert is pending, that timer is lost — the alert stays “pending” until an operator manually acts on it, rather than auto-sending on schedule. Fine for a single-process demo; a production version would need a persistent scheduler instead. Also: the SMS sent to operators is a notification only — replying to it does nothing. Operators act from the dashboard, not by texting back.